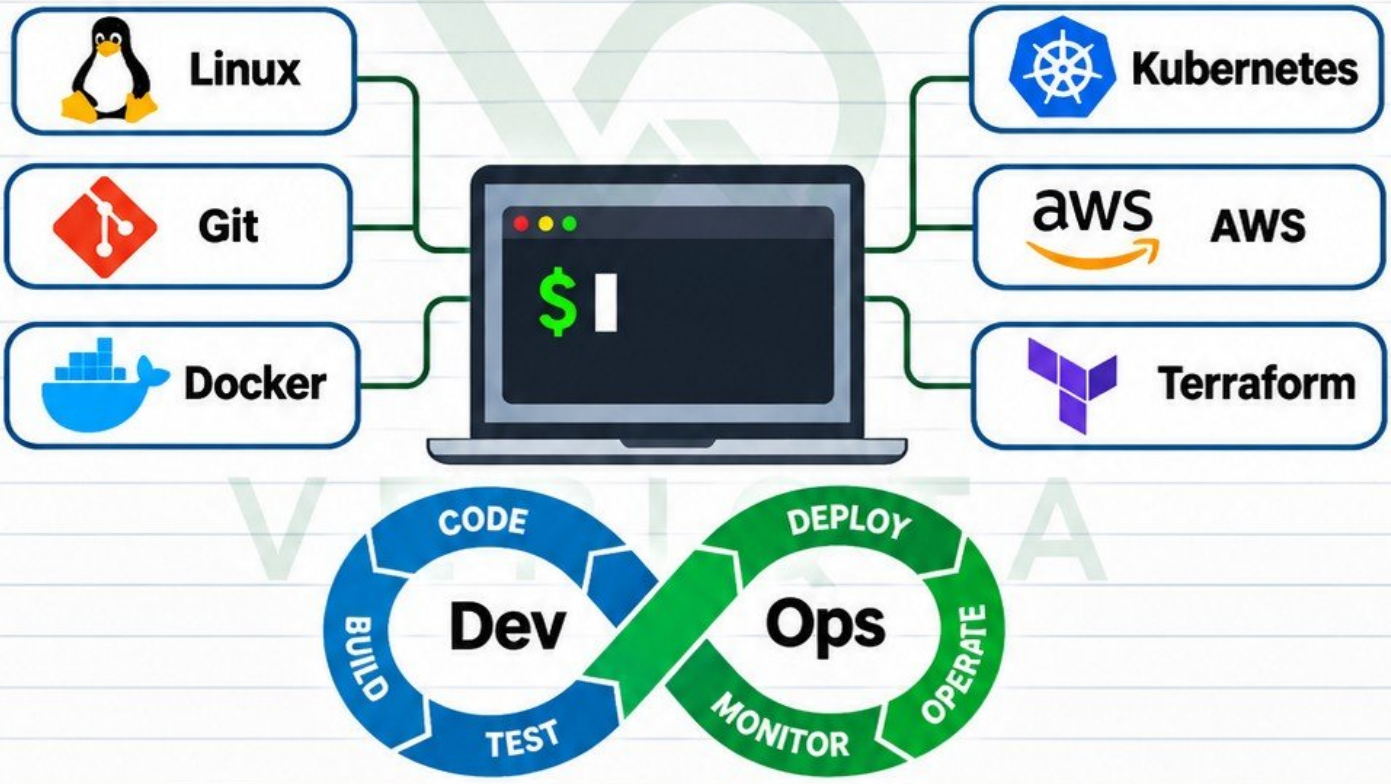
VERIQTA

The DevOps Command Cheat Sheet:

100 Commands You'll Actually Use



- ✓ Linux & Server Operations
- ✓ Logs & Troubleshooting
- ✓ Networking & Connectivity
- ✓ Git & Collaboration
- ✓ Docker & Containers
- ✓ Kubernetes Operations
- ✓ Cloud (AWS) Commands
- ✓ Terraform & Automation
- ✓ Production & Incident Response

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Know Your Server Before You Touch It

Goal: Identify who you are, which server you are on, the OS and kernel version, how long it has been running, and its memory health before making any changes.

1	whoami	Identify your current user and privilege context.	`$ whoami` `ubuntu`

2	hostnamectl	Inspect the server hostname, operating system, and architecture.	`$ hostnamectl`
			`Static hostname: prod-web01`
			`Operating System: Ubuntu 24.04 LTS`
			`Architecture: x86_64`

3	uname -r	Check the running Linux kernel version.	`$ uname -r` `6.8.0-52-generic`

4	uptime	Check how long the server has been running and its load averages.	`$ uptime`
			`10:24:30 up 12 days, 4:18,`
			`2 users, load average: 0.15, 0.20, 0.12`

5	free -h	Inspect available and used system memory.	`$ free -h`
			`total used free`
			`Mem: 7.7G 2.4G 5.1G`



Real-world Scenario: You have just connected to an unfamiliar production server. Establish its identity and basic health before making changes.

Junior Engineer Tip: Never assume you are connected to the correct server or operating with the intended privileges.

Navigate Files Without Breaking Anything

Goal: Move around the Linux filesystem, inspect files safely, and find the right configuration files when troubleshooting.

6	`pwd`	Confirm your current working directory.	`$ pwd` `/home/ubuntu`

7	`ls -lah`	Inspect files, hidden entries, ownership, and sizes.	`$ ls -lah`
			`total 24K`
			`drwxr-xr-x 4 ubuntu ubuntu 4.0K .`
			`-rw-r--r-- 1 ubuntu ubuntu 8.2K app.log`

8	`cd /var/log`	Navigate to the system log directory.	`$ cd /var/log && pwd` `/var/log`

9	`find /etc -name "*.conf"`	Locate configuration files.	`$ find /etc -name "*.conf"`
			`/etc/nginx/nginx.conf`
			`/etc/ssh/sshd_config`

10	`stat /etc/hosts`	Inspect file metadata, permissions, and timestamps.	`$ stat /etc/hosts`
			`Size: 403 Perm: -rw-r--r--`
			`Owner: root Group: root`
			`Modify: 2026-11-02 10:24:31`



Real-world Scenario: Locate the configuration file responsible for an application startup failure.

Beginner Mistake: Confusing the current working directory with the directory containing the application.

Understand Permissions Before Using sudo

Goal: Understand Linux users, groups, and file permissions so you can fix access issues safely and use sudo correctly when needed.

11 id

Inspect user IDs and group membership.

```
$ id
uid=1000(ubuntu)
gid=1000(ubuntu)
groups=1000(ubuntu),4(adm)
```

12 ls -l

Examine file ownership and permissions.

```
$ ls -l app.conf
-rw-r--r-- 1 ubuntu ubuntu
2.1K Nov 2 10:24 app.conf
```

13 namei -l /path/to/file

Inspect permissions along a file path (directory by directory).

```
$ namei -l /etc/ssh/sshd_config
f: /etc/ssh/sshd_config
d  /           drwxr-xr-x  root root
d  etc         drwxr-xr-x  root root
d  ssh         drwxr-xr-x  root root
-  sshd_config -rw-r--r--  root root
```

14 chmod 600 private.key

Restrict a private key to owner read/write access.

```
$ chmod 600 private.key
$ ls -l private.key
-rw------- 1 ubuntu ubuntu 1.6K
Nov 2 10:30 private.key
```

15 sudo -l

Check which privileged commands your account may run.

```
$ sudo -l
User ubuntu may run the
following commands on this host:
(ALL : ALL) ALL
```



Real-world Scenario: Investigate why an application cannot read its configuration file or why SSH rejects a private key.



Security Reminder: Do not use **chmod 777** as a universal permission fix. It gives read, write, and execute access to everyone.

Investigate Processes and Failed Services

Goal: Check running processes, monitor resource usage, and investigate a service failure using systemd and logs.

16 `ps aux` — Inspect running processes.

```
$ ps aux | head -n 3
USER       PID  %CPU  %MEM    COMMAND
root         1   0.0   0.2    systemd
nginx     2543   0.1   0.5    nginx
```

17 `top` — Monitor live CPU and memory usage.

```
$ top
CPU(s):  5.2%us,  1.0%sy,  0.0%id
Mem:   3.9G total,  1.2G used,
       2.7G free
```

18 `systemctl status nginx` — Check a service's state and recent diagnostics.

```
$ systemctl status nginx
● nginx.service - active (running)
   Loaded: enabled
   Main PID: 2543 (nginx)
```

19 `journalctl -u nginx -n 50` — Inspect the recent service logs.

```
$ journalctl -u nginx -n 50
Nov 02 10:18:24 nginx[2543]:
server: started
Nov 02 10:20:01 nginx[2543]:
error: open() failed ...
```

20 `systemctl is-enabled nginx` — Check whether a service is configured to start automatically.

```
$ systemctl is-enabled nginx
enabled
```

Real-world Scenario: An Nginx web server stops responding. Determine whether the process is running and whether systemd reports a failure.

Troubleshooting Tip: Investigate the failure (e.g., process status, logs, and error messages) before restarting the service.

Read Logs Like an Incident Responder

Goal: Use log commands to quickly find errors, understand what happened, and identify the cause of an issue.

21 `tail -n 50 application.log`

Read the last 50 lines of an application log.

```
$ tail -n 50 application.log
[10:14:21] INFO  Server started
[10:15:03] ERROR Failed to
connect to database
```

22 `tail -f application.log`

Follow new log entries as they arrive (real time).

```
$ tail -f application.log
[10:16:02] INFO  Health check OK
[10:16:10] ERROR HTTP 500
on /api/users
```

23 `grep -n "ERROR" application.log`

Find errors and display their line numbers.

```
$ grep -n "ERROR" application.log
120:[ERROR] Failed to connect
256:[ERROR] HTTP 500 on /api/users
```

24 `grep -C 3 "timeout" application.log`

Show 3 lines before and after a timeout message (context).

```
$ grep -C 3 "timeout" application.log
197-INFO  Sending request...
198-INFO  Waiting for response...
199-ERROR timeout after 30s
200-INFO  Retrying (1/3)
201-INFO  Request failed
```

25 `awk '{print $1}' access.log`

Extract the first field from each line (e.g., client IP in an access log).

```
$ awk '{print $1}' access.log
192.168.1.10
192.168.1.11
10.0.0.5
```



Real-world Scenario: Investigate an application returning HTTP 500 errors using log analysis.

Troubleshooting Tip: When analyzing logs, look for error keywords, timestamps, and patterns that point to the root cause.

Diagnose Disk and Filesystem Problems

Goal: Use the right commands to check disk usage, inodes, and filesystem details when troubleshooting storage issues.

26 df -h

Inspect filesystem capacity.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        20G  8.1G   11G  43% /
tmpfs           1.9G     0  1.9G   0% /dev/shm
/dev/sda2       100G   12G   88G  12% /data
```

27 df -i

Detect inode exhaustion.

```
$ df -i
Filesystem     Inodes  IUsed  IFree IUse% Mounted on
/dev/sda1      524288 520000   4288   99% /
tmpfs           49152   1024  48128    3% /dev/shm
```

28 du -sh /var/log

Measure disk space used by logs.

```
$ du -sh /var/log
2.4G    /var/log
```

29 lsblk -f

Inspect block devices, filesystems, and mount points.

```
$ lsblk -f
NAME   FSTYPE   LABEL   UUID   MOUNTPOINT
sda    ext4     -       -      /
sda1   ext4     -       -      /boot
sdb    xfs      data    -      /data
```

30 findmnt

Examine mounted filesystems.

```
$ findmnt
TARGET   SOURCE     FSTYPE  OPTIONS
/        /dev/sda1  ext4    rw,relatime
/boot    /dev/sda1  ext4    rw,relatime
/data    /dev/sdb   xfs     rw,relatime
```



Real-world Scenario: An application cannot write logs even though the server appears healthy.



Common Failure: A filesystem can run out of inodes even when storage space remains available.

Troubleshoot DNS and Network Connectivity

Goal: Diagnose network connectivity issues by checking network interfaces, routing, DNS resolution, and reachability between systems.

31 `ip addr` — Inspect network interfaces and IP addresses.

```
$ ip addr
2: eth0: <UP,BROADCAST,RUNNING>
   inet 10.0.1.15/24
   inet6 fe80::a00:27ff:fe4e:1/64
```

32 `ip route` — Examine routing information.

```
$ ip route
default via 10.0.1.1 dev eth0
10.0.1.0/24 dev eth0 proto kernel
```

33 `dig example.com` — Query DNS records for a domain.

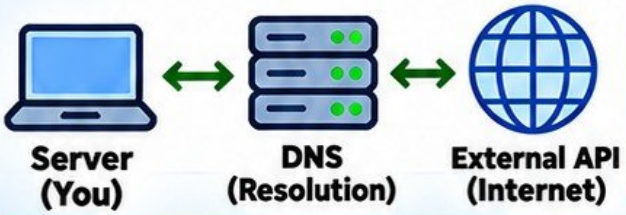
```
$ dig example.com
;; ANSWER SECTION:
example.com.    300   IN   A
 93.184.216.34
```

34 `ping -c 4 1.1.1.1` — Test ICMP reachability (with 4 packets).

```
$ ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes
--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% loss
```

35 `traceroute example.com` — Inspect the network path to a destination.

```
$ traceroute example.com
 1   10.0.1.1          1.2 ms
 2   41.220.0.1        5.6 ms
 3   93.184.216.34     18.9 ms
```



Real-world Scenario: A server cannot reach an external API. Determine whether the problem involves DNS, routing, or connectivity.

Junior Engineer Tip: Failed ping responses do not automatically prove that a server is offline. ICMP may be blocked by a firewall or security policy.

Debug Ports, HTTP, and TLS

Goal: Troubleshoot service access by checking ports, HTTP responses, and TLS (HTTPS) connectivity and certificates.

36 `ss -tulpen`

Inspect listening TCP/UDP sockets and associated processes.

```
$ ss -tulpen | grep :80
tcp  LISTEN  0  511  *:80
users:(("nginx",pid=2543,fd=6))
```

37 `curl -I https://example.com`

Request HTTP response headers.

```
$ curl -I https://example.com
HTTP/1.1 200 OK
Server: nginx
Content-Type: text/html
Content-Length: 6159
```

38 `curl -v https://example.com`

Inspect the request, connection, and TLS diagnostics.

```
$ curl -v https://example.com
* TLSv1.3 (IN), TLS handshake, ...
* SSL connection using TLSv1.3
< HTTP/1.1 200 OK
```

39 `nc -vz example.com 443`

Test whether a TCP connection can be established (to a specific port).

```
$ nc -vz example.com 443
Connection to example.com 443
succeeded!
```

40 `openssl s_client -connect example.com:443 -servername example.com`

Inspect a TLS connection and the certificate chain.

```
$ openssl s_client -connect
example.com:443 -servername example.com
CONNECTED(00000003)
Certificate chain
 0 s:CN = example.com
 Verify return code: 0 (ok)
```



Real-world Scenario: A website is unavailable despite the application service running.

Troubleshooting Focus: Distinguish DNS failures, connection refusals, timeouts, HTTP errors, and certificate problems.

Work Safely With Git Repositories

Goal: Use essential Git commands to manage code changes safely, keep a clean history, and avoid accidental commits.

41 git clone <repo-url>

Obtain a local repository.

```
$ git clone https://github.com/veriqta/app-config.git
Cloning into 'app-config'...
remote: Enumerating objects: 128...
Receiving objects: 100% (128/128).
```

42 git status

Inspect changed, staged, and untracked files.

```
$ git status
On branch main
Changes not staged for commit:
  modified:   deploy.yaml
Untracked files:
  notes.txt
```

43 git diff

Review unstaged changes.

```
$ git diff
--- a/deploy.yaml
+++ b/deploy.yaml
@@ -3,7 +3,7 @@
- replicas: 1
+ replicas: 2
```

44 git add <file>

Stage a specific file.

```
$ git add deploy.yaml
$ git status
Changes to be committed:
  modified:   deploy.yaml
```

45 git commit -m "Describe change"

Record a reviewed change locally.

```
$ git commit -m "Increase replicas to 2"
[main 3f2a1c4] Increase replicas to 2
 1 file changed, 1 insertion(+),
 1 deletion(-)
```

Real-world Scenario: Update an application's deployment configuration while preserving a clear history.

Beginner Mistake: Staging everything without checking for secrets or unrelated changes.

Collaborate Without Damaging Shared Code

Goal: Use Git collaboration workflows to work on new features safely, keep code in sync, and prepare changes for review and merge.

46 `git switch -c feature/health-check`

Create and switch to a feature branch.

```
$ git switch -c feature/health-check
Switched to a new branch 'feature/health-check'
```

47 `git fetch origin`

Retrieve remote changes without merging them.

```
$ git fetch origin
From github.com:veriqta/app
 * [new commits]     main -> origin/main
 * [new branch]      feature/login -> origin/feature/login
```

48 `git log --oneline --graph -10`

Review the recent commit history (last 10).

```
$ git log --oneline --graph -10
* 7f3a2c1 Add health check endpoint
* 4e9b7d0 Update dependencies
|\
| * 2c1d8a9 Fix logging issue
|/
* 9a6f4e2 Initial app setup
```

49 `git diff origin/main...HEAD`

Review changes introduced by the current branch relative to the merge base.

```
$ git diff origin/main...HEAD
diff --git a/healthcheck.py
+++ b/healthcheck.py
+ def health_check():
+     return {"status": "ok"}
```

50 `git push -u origin feature/health-check`

Publish the feature branch to the remote repository.

```
$ git push -u origin feature/health-check
Enumerating objects: 5, done.
Writing objects: 100% (5/5), 512 bytes
To github.com:veriqta/app.git
 * [new branch]   feature/health-check -> feature/health-check
Branch 'feature/health-check' set up to track 'origin/feature/health-check'.
```

Real-world Scenario: Prepare a health-check change for peer review through a pull request.

Production Awareness: Shared branches should use reviews and automated checks (e.g., CI) before merging.

Automate Routine Server Tasks With Bash

Goal: Use essential Bash and scheduling commands to debug scripts and verify scheduled tasks on Linux servers.

51 bash -n deploy.sh

Check Bash script syntax without executing it.

```
$ bash -n deploy.sh
(no output means syntax is OK)
$ echo $?
0
```

52 bash -x deploy.sh

Trace script execution for debugging.

```
$ bash -x deploy.sh
+ set -e
+ echo 'Starting deployment...'
+ ./install.sh
+ echo 'Deployment completed.'
```

53 shellcheck deploy.sh

Detect common shell scripting problems.

```
$ shellcheck deploy.sh
In deploy.sh line 12:
SC2086: Double quote to prevent
globbing and word splitting.
```

54 crontab -l

Inspect the current user's scheduled cron jobs.

```
$ crontab -l
# m h dom mon dow   command
0 2 * * * /home/ubuntu/backup.sh
30 3 * * 1 /home/ubuntu/cleanup.sh
```

55 systemctl list-timers --all

Inspect systemd timers and their schedules.

```
$ systemctl list-timers --all
NEXT                         LEFT     UNIT
Sat 2026-11-04 02:00:00      5h       backup.timer
Sat 2026-11-04 03:00:00      6h       logrotate.timer
Sun 2026-11-05 00:00:00      1 day    apt-daily.timer
Sun 2026-11-05 01:00:00      1 day    apt-daily-upgrade.timer
```

Real-world Scenario: Investigate why a scheduled backup or maintenance script is not executing.

Security Reminder: Bash tracing may expose sensitive variable values. Do not enable it carelessly around secrets.

Understand Docker Images and Containers

Goal: Use key Docker commands to inspect images, containers, and logs to diagnose container issues.

56 docker version

Check Docker client and server versions.

```
$ docker version
Client: Docker Engine - Community
  Version: 28.3.1
Server: Docker Engine - Community
  Version: 28.3.1
```

57 docker image ls

List locally available container images.

```
$ docker image ls
REPOSITORY   TAG       IMAGE ID       SIZE
nginx        latest    7e2d2a1f5c3b   192MB
myapp        v1.0      3ac4b9e7d8f1   285MB
```

58 docker ps -a

Inspect running and stopped containers.

```
$ docker ps -a
CONTAINER ID   IMAGE          STATUS
a1b2c3d4e5f6   myapp:v1.0     Exited (1) 8s ago
9f8e7d6c5b4a   nginx:latest   Up 2 hours
```

59 docker inspect <container>

Examine container configuration and runtime state.

```
$ docker inspect myapp
{
    "Id": "a1b2c3d4e5f6",
    "Image": "myapp:v1.0",
    "State": {"Status": "exited",
    "ExitCode": 1}
  ...
}
```

60 docker logs --tail 50 <container>

Read recent container logs.

```
$ docker logs --tail 50 myapp
2026-11-03 10:12:01 Starting app...
2026-11-03 10:12:02 ERROR: Missing
required environment variable: DB_URL
2026-11-03 10:12:02 Exiting (1)
```



Real-world Scenario: A deployed container exits immediately after starting.



Beginner Mistake: Confusing an image, which packages the application, with a container, which is a runtime instance.

Run and Troubleshoot Containerized Applications

Goal: Run containerized applications, inspect their configuration, and troubleshoot common access and runtime issues.

61 `docker run -d --name web -p 8080:80 nginx:stable`

Start an Nginx container with a published port.

```
$ docker run -d --name web -p 8080:80 nginx:stable
9f8e7d6c5b4a2d1f0e3a9c7b8e2d6f1a
3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8
```

62 `docker exec -it web sh`

Open a shell in the running container.

```
$ docker exec -it web sh
/ # nginx -v
nginx version: nginx/1.28.0
/ #
```

63 `docker stats --no-stream`

Inspect container resource consumption.

```
$ docker stats --no-stream
CONTAINER   CPU %   MEM USAGE / LIMIT
web         0.85%   12.3MiB / 512MiB
```

64 `docker port web`

Display published container ports.

```
$ docker port web
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

65 `docker compose ps`

Inspect services managed by Docker Compose.

```
$ docker compose ps
NAME   IMAGE          STATE     PORTS
web    nginx:stable   running   0.0.0.0:8080->80/tcp
db     postgres:16    running   5432/tcp
```

Real-world Scenario: An application works inside its container but cannot be reached from the host.

Production Awareness: Publishing a port can expose a service beyond localhost, depending on host networking and firewall configuration.

Inspect a Kubernetes Cluster Before Deployment

Goal: Inspect and understand a Kubernetes cluster, its nodes, namespaces, and workloads before making any changes.

66 kubectl config current-context

Identify the selected Kubernetes context.

```
$ kubectl config current-context
prod-cluster
```

67 kubectl cluster-info

Check cluster endpoint information.

```
$ kubectl cluster-info
Kubernetes control plane is running at
https://10.0.0.10:6443
CoreDNS is running at
https://10.0.0.10:6443/api/v1/namespaces/
kube-system/services/kube-dns:dns
```

68 kubectl get nodes -o wide

Inspect nodes and their status.

```
$ kubectl get nodes -o wide
NAME       STATUS   ROLES    AGE   INTERNAL-IP
worker-1   Ready    <none>   12d   10.0.1.11
worker-2   Ready    <none>   12d   10.0.1.12
```

69 kubectl get namespaces

List namespaces in the cluster.

```
$ kubectl get namespaces
NAME          STATUS   AGE
default       Active   12d
kube-system   Active   12d
monitoring    Active   10d
production    Active    5d
```

70 kubectl get pods -A

Inspect Pods across all namespaces.

```
$ kubectl get pods -A
NAMESPACE     NAME                READY   STATUS    AGE
kube-system   coredns-6d4b7       1/1     Running   12d
monitoring    prometheus-0        1/1     Running   10d
production    web-7f8d9c5d7b      2/2     Running    5d
```

Real-world Scenario: You receive a Kubernetes access configuration and need to understand the environment before making changes.

Security Reminder: Always verify your context and namespace before modifying Kubernetes resources.

Verify Deployments, Services, and Rollouts

Goal: Verify the health and connectivity of Deployments, Services, and rollouts to confirm that a new application version is running and reachable.

76 `kubectl get deployments -n <namespace>`

Inspect application Deployments in a namespace.

```
$ kubectl get deployments -n prod
NAME      READY   UP-TO-DATE   AVAILABLE
web-app   3/3     3            3
api       2/2     2            2
```

77 `kubectl get svc -n <namespace>`

Inspect Kubernetes Services in a namespace.

```
$ kubectl get svc -n prod
NAME      TYPE        CLUSTER-IP     PORT(S)
web-svc   ClusterIP   10.96.20.15    80/TCP
api-svc   ClusterIP   10.96.20.27    8080/TCP
```

78 `kubectl get endpointslices -n <namespace>`

Examine Service backend endpoint information.

```
$ kubectl get endpointslices -n prod
NAME              ADDRESS TYPE   ENDPOINTS
web-svc-8f6d2     IPv4           10.1.0.12,10.1.0.13
api-svc-a7c9e     IPv4           10.1.1.7,10.1.1.8
```

79 `kubectl rollout status deployment/<name> -n <namespace>`

Monitor rollout completion for a Deployment.

```
$ kubectl rollout status \
  deployment/web-app -n prod
deployment "web-app" successfully
rolled out
```

80 `kubectl rollout history deployment/<name> -n <namespace>`

Review available rollout revisions for a Deployment.

```
$ kubectl rollout history \
  deployment/web-app -n prod
REVISION    CHANGE-CAUSE
1           <none>
2           Update image to v1.2.0
```

Real-world Scenario: A new application version is deployed, but users cannot reach it.

Junior Engineer Tip: A healthy Deployment does not automatically mean that its Service is routing traffic to ready application endpoints.

Investigate AWS Resources From the CLI

Goal: Use the AWS CLI to verify your identity, configuration, and the state of key resources such as EC2, security groups, and CloudWatch alarms for troubleshooting.

81 aws sts get-caller-identity

Verify the active AWS identity (account and principal).

```
$ aws sts get-caller-identity
{
  "UserId": "AIDAEXAMPLE123456",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/veriqta"
}
```

82 aws configure list

Inspect CLI configuration sources and selected settings.

```
$ aws configure list
Name      Value      Type      Location
profile   default    config    ~/.aws/config
region    us-east-1  config    ~/.aws/config
output    json       config    ~/.aws/config
```

83 aws ec2 describe-instances --region us-east-1

Inspect EC2 instance information in the selected region.

```
$ aws ec2 describe-instances --region us-east-1
{
   "Reservations": [{
      "Instances": [{
         "InstanceId": "i-0abc123def4567890",
         "State": {"Name": "running"},
         "InstanceType": "t3.micro"
   } ]
}
```

84 aws ec2 describe-security-groups --region us-east-1

Examine security group configurations (inbound and outbound rules).

```
$ aws ec2 describe-security-groups --region us-east-1
{
   "SecurityGroups": [{
      "GroupId": "sg-0a1b2c3d4e5f67890",
      "GroupName": "web-sg",
      "IpPermissions": [{ "FromPort": 80, "ToPort": 80,
   "IpProtocol": "tcp", "IpRanges": [{"CidrIp": "0.0.0.0/0"}] }]
}]
```

85 aws cloudwatch describe-alarms --region us-east-1

Inspect CloudWatch alarm definitions and their current states.

```
$ aws cloudwatch describe-alarms --region us-east-1
{
   "MetricAlarms": [{
      "AlarmName": "EC2-High-CPU",
      "StateValue": "OK",
      "Namespace": "AWS/EC2"
   }]
}
```

Real-world Scenario: Investigate an unreachable EC2 instance without changing infrastructure.

Security Reminder: Verify the AWS account, role, and region before executing cloud commands.

Validate Infrastructure Changes With Terraform

Goal: Use Terraform CLI commands to check, validate, and preview infrastructure changes before applying them to your cloud environment.

86 terraform version

Inspect the installed Terraform version.

```
$ terraform version
Terraform v1.8.5
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws
v5.68.0
```

87 terraform init

Initialize the working directory and providers.

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Using hashicorp/aws v5.68.0
Terraform has been successfully initialized!
```

88 terraform fmt -check -recursive

Check Terraform formatting across files.

```
$ terraform fmt -check -recursive
All Terraform configuration files
are properly formatted.
```

89 terraform validate

Validate the Terraform configuration for errors.

```
$ terraform validate
Success! The configuration is valid.
```

90 terraform plan

Preview proposed infrastructure changes.

```
$ terraform plan
Plan: 2 to add, 1 to change, 0 to destroy.
+ aws_subnet.public (new)
~ aws_security_group.web (update)
+ aws_route_table.pub (new)
```

Terraform

Real-world Scenario: Review a change to cloud networking infrastructure before applying it.

Production Awareness: A successful plan does not guarantee a risk-free change. Review possible resource replacements, deletions, and permissions before applying.

Investigate Performance and Resource Pressure

Goal: Use Linux performance and diagnostic commands to identify CPU, memory, disk I/O, and process-level resource pressure when an application becomes slow.

91 vmstat 1 5

Sample CPU, memory, and system activity.

```
$ vmstat 1 5
procs -----------memory---------- ----cpu--
r  b   swpd   free   buff  cache   si so  us sy id wa st
0  0      0 524328  62344 1014280  0  0   6  3 89  2  0
1  0      0 523900  62344 1014280  0  0   7  4 87  2  0
```

92 iostat -xz 1 3

Examine disk I/O utilization and latency indicators.

```
$ iostat -xz 1 3
Device    r/s    w/s    rkB/s    wkB/s   %util   await
sda      12.3   45.6    980.0   3650.0    78.5    12.7
sdb       0.4    2.1     24.0    144.0     4.2     1.9
```

93 pidstat -dur 1 3

Inspect per-process CPU, memory, and disk activity.

```
$ pidstat -dur 1 3
 PID   %usr %sys %CPU  %mem  kB_rd/s kB_wr/s  Command
3124  18.2  4.1 22.3   6.4     0.00   12.00  java
4287   2.1  1.5  3.6   1.2     0.00    0.00  nginx
```

94 dmesg -T

Inspect kernel messages with human-readable timestamps.

```
$ dmesg -T | tail -3
[Tue Nov 03 17:20:41 2026] CPU: Core temp
 above threshold, throttling.
[Tue Nov 03 17:20:44 2026] nvme0: I/O timeout,
 resetting controller.
[Tue Nov 03 17:20:47 2026] systemd: Started
 Cleanup of Temporary Directories.
```

95 lsof -iTCP -sTCP:LISTEN -n -P

Identify processes listening on TCP ports.

```
$ lsof -iTCP -sTCP:LISTEN -n -P
COMMAND   PID  USER   FD   TYPE  LOCAL ADDRESS
nginx    4287  www    6u   IPv4  0.0.0.0:80 (LISTEN)
java     3124  app    7u   IPv4  0.0.0.0:8080 (LISTEN)
sshd     1098  root   3u   IPv4  0.0.0.0:22 (LISTEN)
```

Real-world Scenario: An application becomes slow while the server remains reachable.

Troubleshooting Focus: Determine whether CPU, memory pressure, disk I/O, or a specific process contributes to the symptoms.

Production Challenge: Find the Failure Before You Fix It

Goal: Use production-grade Linux, systemd, logs, HTTP checks, and Git to identify the root cause of a live application failure before making any changes.

96 date -u

Establish the current UTC time for correlating evidence.

```
$ date -u
Tue, 03 Nov 2026 16:25:42 UTC
```

97 systemctl --failed

Identify failed systemd units.

```
$ systemctl --failed
UNIT           LOAD   ACTIVE SUB    DESCRIPTION
app.service    loaded failed failed Application Service
nginx.service  loaded failed failed Nginx Web Server
2 loaded units listed.
```

98 journalctl -p err -b

Review error-priority messages from the current boot.

```
$ journalctl -p err -b | tail -3
Nov 03 16:10:24 app[3241]: ERROR: Failed to
connect to database: connection refused
Nov 03 16:10:24 systemd[1]: app.service:
Main process exited, code=exited, status=1/FAILURE
```

99 curl -sS -o /dev/null -w '%{http_code}' http://127.0.0.1:8080/health

Check an application's local HTTP health endpoint.

```
$ curl -sS -o /dev/null -w '%{http_code}W
 http://127.0.0.1:8080/health
503
```

100 git show --stat --oneline HEAD

Inspect the latest commit summary and affected files.

```
$ git show --stat --oneline HEAD
a1b2c3d  Deploy v2.4.0: update config
and health check
 3 files changed, 42 insertions(+),
 9 deletions(-)
  config/app.yaml  |  24 +++++++--
  src/health.py    |  15 ++++++
  README.md        |   3 ++-
```

Final Practical Scenario: A deployment has just completed. The application is returning HTTP 503 responses, users cannot access it, and the monitoring dashboard reports increased failures.